MTM4692

Spring 2026

Fettah KIRAN
Department of AI and Data Engineering
fkiran@yildiz.edu.tr

Functions & Stored Procedures

MySQL Functions, Procedures & SQLite Equivalents

📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:05	Recap Quiz	Review
1:05 – 1:25	Functions vs Procedures	**Theory** 📖
1:25 – 1:55	Creating Functions	**Theory** 📖
1:55 – 2:15	Creating Stored Procedures	**Theory** 📖
2:15 – 2:30	☕ Break (15 min)	
2:30 – 2:50	Parameters: IN, OUT, INOUT	**Theory** 📖
2:50 – 3:05	DETERMINISTIC vs NOT DETERMINISTIC	**Theory** 📖
3:05 – 3:20	SQLite Equivalents	**Practice** 💻
3:20 – 3:40	Practice & Homework	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

Learning Objectives

By the end of this week you will be able to:

1. Explain the **difference** between functions and stored procedures
2. Create **MySQL functions** with RETURNS and RETURN
3. Create **MySQL stored procedures** with IN/OUT/INOUT parameters
4. Use **DELIMITER** to define multi-statement routines
5. Understand **DETERMINISTIC** vs **NOT DETERMINISTIC**
6. Implement **SQLite alternatives** using CTEs and views

Part 1: Functions vs Procedures — Overview

Functions vs Procedures at a Glance

Feature	Function	Stored Procedure
Returns value?	**Yes** (RETURNS)	No (uses OUT params)
Call syntax	`SELECT func()`	`CALL proc()`
Use in SELECT?	✅ Yes	❌ No
Multiple results?	No	Yes (result sets)
DML (INSERT/UPDATE)?	Limited/No	✅ Yes
Transaction control?	❌ No	✅ Yes
Error handling?	Limited	Full (HANDLER)

When to Use Which?

Use a Function when:

- You need a **calculated value** (e.g., tax, discount, grade)
- You want to use it **inside SELECT, WHERE, ORDER BY**
- It's a pure computation with **no side effects**

Use a Stored Procedure when:

- You need to **modify data** (INSERT, UPDATE, DELETE)
- You need **multiple output values**
- You need **transaction control**
- Business logic requires **error handling**

🧠 Exam Focus — Key Distinction

> **Function** = returns a single value, callable from SQL expressions
>
> **Procedure** = performs actions, called with CALL, can return multiple values via OUT parameters

Part 2: Creating MySQL Functions

Basic Function Syntax

```
DELIMITER //

CREATE FUNCTION function_name(param1 TYPE, param2 TYPE)
RETURNS return_type
DETERMINISTIC

BEGIN
    DECLARE result return_type;
    -- logic here
    SET result = ...;
    RETURN result;
END //

DELIMITER ;
```

Key elements:

- `RETURNS` declares the return **type**
- `RETURN` actually sends the **value** back
- Must specify `DETERMINISTIC` or `NOT DETERMINISTIC`

Example: Letter Grade Function

```
DELIMITER //
CREATE FUNCTION get_letter_grade(p_gpa DECIMAL(3,2))
RETURNS VARCHAR(2)
DETERMINISTIC
BEGIN
    DECLARE grade VARCHAR(2);
    IF p_gpa >= 3.7 THEN SET grade = 'A';
    ELSEIF p_gpa >= 3.3 THEN SET grade = 'B+';
    ELSEIF p_gpa >= 3.0 THEN SET grade = 'B';
    ELSEIF p_gpa >= 2.7 THEN SET grade = 'C+';
    ELSEIF p_gpa >= 2.0 THEN SET grade = 'C';
    ELSEIF p_gpa >= 1.0 THEN SET grade = 'D';
    ELSE SET grade = 'F';
    END IF;

    RETURN grade;
END //

DELIMITER ;
```

Using the Function

```sql
-- In SELECT
SELECT first_name, gpa, get_letter_grade(gpa) AS grade
FROM student;

-- In WHERE
SELECT first_name, gpa
FROM student
WHERE get_letter_grade(gpa) = 'A';

-- In ORDER BY
SELECT first_name, gpa, get_letter_grade(gpa) AS grade
FROM student
ORDER BY get_letter_grade(gpa);
```

Functions behave like **built-in functions** (UPPER, ROUND, etc.)

Example: Discount Calculator

```
DELIMITER //

CREATE FUNCTION calc_discount(
    p_total DECIMAL(10,2),
    p_type VARCHAR(20)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE discount DECIMAL(10,2);

    CASE p_type
        WHEN 'gold'     THEN SET discount = p_total * 0.20;
        WHEN 'silver'   THEN SET discount = p_total * 0.10;
        WHEN 'bronze'   THEN SET discount = p_total * 0.05;
        ELSE SET discount = 0.00;
    END CASE;

    RETURN discount;
END //

DELIMITER ;
```

Using Discount Function

```sql
SELECT
    customer_name,
    order_total,
    membership_type,
    calc_discount(order_total, membership_type) AS discount,
    order_total - calc_discount(order_total, membership_type) AS final
FROM orders
JOIN customer USING (customer_id);
```

The function is used just like any **expression** in SQL!

Example: Age Calculator

```
DELIMITER //

CREATE FUNCTION calc_age(p_birthdate DATE)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN TIMESTAMPDIFF(YEAR, p_birthdate, CURDATE());
END //

DELIMITER ;
```

```
SELECT first_name, birthdate,
       calc_age(birthdate) AS age
FROM student;
```

🧠 Exam Focus — Function Rules

1. Functions must have **RETURNS** (return type declaration)
2. Functions must have at least one **RETURN** statement
3. Functions can be used in **SELECT, WHERE, ORDER BY**
4. Functions should **NOT** modify data (no INSERT/UPDATE/DELETE)
5. Must specify **DETERMINISTIC** or **NOT DETERMINISTIC**

Managing Functions

```sql
-- List all functions in a database
SHOW FUNCTION STATUS WHERE Db = 'university';

-- See function definition
SHOW CREATE FUNCTION get_letter_grade;

-- Drop a function
DROP FUNCTION IF EXISTS get_letter_grade;

-- Modify: must DROP and re-CREATE
DROP FUNCTION IF EXISTS calc_discount;
-- then CREATE FUNCTION ...
```

Part 3: Creating Stored Procedures

Basic Procedure Syntax

```
DELIMITER //
CREATE PROCEDURE procedure_name(
    IN param1 TYPE,
    OUT param2 TYPE,
    INOUT param3 TYPE
)
BEGIN
    -- SQL statements
    -- Can include SELECT, INSERT, UPDATE, DELETE
    -- Can include flow control, error handling
END //
DELIMITER ;
```

- No `RETURNS` clause — use **OUT** parameters instead
- Called with `CALL procedure_name(...)`

Example: Simple Query Procedure

```
DELIMITER //

CREATE PROCEDURE get_students_by_dept(
    IN p_dept_id INT
)
BEGIN
    SELECT s.student_id, s.first_name, s.last_name, s.gpa
    FROM student s
    WHERE s.dept_id = p_dept_id
    ORDER BY s.gpa DESC;
END //

DELIMITER ;
```

```
-- Call it
CALL get_students_by_dept(1);   -- CS students
CALL get_students_by_dept(2);   -- Math students
```

Example: Procedure with OUT Parameter

```
DELIMITER //

CREATE PROCEDURE get_dept_stats(
    IN  p_dept_id   INT,
    OUT p_count     INT,
    OUT p_avg_gpa   DECIMAL(3,2),
    OUT p_max_gpa   DECIMAL(3,2)
)
BEGIN
    SELECT COUNT(*), AVG(gpa), MAX(gpa)
    INTO p_count, p_avg_gpa, p_max_gpa
    FROM student
    WHERE dept_id = p_dept_id;
END //

DELIMITER ;
```

Using OUT Parameters

```sql
-- Call with user variables to receive output
CALL get_dept_stats(1, @cnt, @avg, @max);

-- Access the results
SELECT @cnt AS student_count,
       @avg AS avg_gpa,
       @max AS max_gpa;
```

Output:

student_count	avg_gpa	max_gpa
15	3.24	3.95

Example: INOUT Parameter

```
DELIMITER //

CREATE PROCEDURE apply_curve(
    INOUT p_gpa DECIMAL(3,2),
    IN    p_curve DECIMAL(2,2)
)
BEGIN
    SET p_gpa = p_gpa + p_curve;
    IF p_gpa > 4.0 THEN
        SET p_gpa = 4.0;   -- Cap at 4.0
    END IF;
END //

DELIMITER ;

SET @student_gpa = 3.65;
CALL apply_curve(@student_gpa, 0.50);
SELECT @student_gpa;  -- 4.00 (capped)
```

🧠 Exam Focus — Parameter Modes

Mode	Direction	Description
IN	Caller → Procedure	Pass value in (default)
OUT	Procedure → Caller	Return value to caller
INOUT	Both directions	Pass in, modify, return

> If no mode is specified, **IN** is the default!

Example: Enrollment Procedure (DML)

```sql
DELIMITER //

CREATE PROCEDURE enroll_student(
    IN  p_student_id INT,
    IN  p_course_id  INT,
    OUT p_status     VARCHAR(50)
)
BEGIN
    DECLARE v_exists INT DEFAULT 0;

    -- Check if already enrolled
    SELECT COUNT(*) INTO v_exists
    FROM enrollment
    WHERE student_id = p_student_id AND course_id = p_course_id;

    IF v_exists > 0 THEN
        SET p_status = 'Already enrolled';
    ELSE
        INSERT INTO enrollment (student_id, course_id, enroll_date)
        VALUES (p_student_id, p_course_id, CURDATE());
        SET p_status = 'Successfully enrolled';
    END IF;
END //

DELIMITER ;
```

Using the Enrollment Procedure

```sql
-- First enrollment
CALL enroll_student(101, 5, @msg);
SELECT @msg;  -- 'Successfully enrolled'

-- Try again
CALL enroll_student(101, 5, @msg);
SELECT @msg;  -- 'Already enrolled'
```

This is a typical **business logic** pattern:

- Check → Act → Report

Procedure with Error Handling

```
DELIMITER //

CREATE PROCEDURE safe_insert_student(
    IN p_first VARCHAR(50),
    IN p_last  VARCHAR(50),
    IN p_dept  INT,
    IN p_gpa   DECIMAL(3,2)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error: Insert failed' AS message;
    END;

    START TRANSACTION;

    IF p_gpa < 0 OR p_gpa > 4.0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'GPA must be between 0 and 4.0';
    END IF;

    INSERT INTO student (first_name, last_name, dept_id, gpa)
    VALUES (p_first, p_last, p_dept, p_gpa);

    COMMIT;
    SELECT 'Student added successfully' AS message;
END //

DELIMITER ;
```

Managing Procedures

```sql
-- List all procedures
SHOW PROCEDURE STATUS WHERE Db = 'university';

-- See definition
SHOW CREATE PROCEDURE enroll_student;

-- Drop a procedure
DROP PROCEDURE IF EXISTS enroll_student;
```

Part 4: DETERMINISTIC vs NOT DETERMINISTIC

What is DETERMINISTIC?

DETERMINISTIC: Same inputs **always** produce the same output

```
-- DETERMINISTIC: pure math
CREATE FUNCTION circle_area(radius DOUBLE)
RETURNS DOUBLE
DETERMINISTIC
BEGIN
    RETURN PI() * radius * radius;
END;
```

NOT DETERMINISTIC: Output can vary even with same inputs

```sql
-- NOT DETERMINISTIC: depends on current data/time
CREATE FUNCTION get_student_count(p_dept INT)
RETURNS INT
NOT DETERMINISTIC
BEGIN
    RETURN (SELECT COUNT(*) FROM student WHERE dept_id = p_dept);
END;
```

Why Does it Matter?

	DETERMINISTIC	NOT DETERMINISTIC
Same inputs → same output?	✅ Always	❌ May vary
MySQL can cache results?	✅ Yes	❌ No
Performance	Better	Normal
Uses: tables, CURDATE(), RAND()?	❌ No	✅ Yes

🧠 **Exam Rule**: If the function reads from a **table** or uses `NOW(), CURDATE(), RAND()`, it is **NOT DETERMINISTIC**!

🧠 Exam Focus — DETERMINISTIC

DETERMINISTIC = Mathematical/pure function (same input → same output)

NOT DETERMINISTIC = Reads from tables, uses time/random functions

MySQL **requires** you to specify one of these!
If you write `DETERMINISTIC` but the function reads tables, MySQL won't stop you — but the results may be **incorrect** due to caching.

Quick Classification Exercise

Function	DETERMINISTIC?
`calc_tax(price, rate)` → price × rate	✅
`get_avg_gpa(dept_id)` → reads student table	❌
`format_name(first, last)` → CONCAT	✅
`days_until_exam()` → uses CURDATE()	❌
`celsius_to_fahrenheit(c)` → c × 9/5 + 32	✅
`count_enrollments(sid)` → reads enrollment	❌

Part 5: Advanced Function Examples

Function with Multiple Logic Branches

```
DELIMITER //

CREATE FUNCTION scholarship_amount(
    p_gpa DECIMAL(3,2),
    p_year INT
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE base DECIMAL(10,2);
    DECLARE multiplier DECIMAL(3,2);

    -- Base amount by GPA
    IF p_gpa >= 3.8 THEN SET base = 5000;
    ELSEIF p_gpa >= 3.5 THEN SET base = 3000;
    ELSEIF p_gpa >= 3.0 THEN SET base = 1000;
    ELSE SET base = 0;
    END IF;

    -- Year multiplier
    SET multiplier = CASE p_year
        WHEN 1 THEN 1.0
        WHEN 2 THEN 1.1
        WHEN 3 THEN 1.2
        WHEN 4 THEN 1.3
        ELSE 1.0
    END;

    RETURN base * multiplier;
END //
```

Using the Scholarship Function

```sql
SELECT first_name, gpa, year_of_study,
       scholarship_amount(gpa, year_of_study) AS award
FROM student
WHERE scholarship_amount(gpa, year_of_study) > 0
ORDER BY scholarship_amount(gpa, year_of_study) DESC;
```

first_name	gpa	year_of_study	award
Alice	3.95	3	6000.00
Bob	3.80	2	5500.00
Carol	3.60	4	3900.00

String Processing Function

```
DELIMITER //
CREATE FUNCTION proper_case(p_str VARCHAR(255))
RETURNS VARCHAR(255)
DETERMINISTIC
BEGIN
    DECLARE result VARCHAR(255);
    DECLARE i INT DEFAULT 1;
    DECLARE prev_char CHAR(1) DEFAULT ' ';

    SET result = LOWER(p_str);

    WHILE i <= LENGTH(result) DO
        IF prev_char = ' ' OR prev_char = '-' THEN
            SET result = CONCAT(
                LEFT(result, i - 1),
                UPPER(SUBSTRING(result, i, 1)),
                SUBSTRING(result, i + 1)
            );
        END IF;
        SET prev_char = SUBSTRING(result, i, 1);
        SET i = i + 1;
    END WHILE;

    RETURN result;
END //
DELIMITER ;
```

Procedure with Multiple Result Sets

```sql
DELIMITER //
CREATE PROCEDURE department_report(IN p_dept_id INT)
BEGIN
    -- Result Set 1: Department info
    SELECT dept_name, building, budget
    FROM department WHERE dept_id = p_dept_id;

    -- Result Set 2: Students in department
    SELECT first_name, last_name, gpa
    FROM student WHERE dept_id = p_dept_id
    ORDER BY gpa DESC;

    -- Result Set 3: Courses offered
    SELECT course_name, credits
    FROM course WHERE dept_id = p_dept_id
    ORDER BY course_name;

    -- Result Set 4: Summary statistics
    SELECT COUNT(*) AS students,
           AVG(gpa) AS avg_gpa,
           (SELECT COUNT(*) FROM course
            WHERE dept_id = p_dept_id) AS courses
    FROM student WHERE dept_id = p_dept_id;
END //
DELIMITER ;
```

Procedure: Batch Update with Logging

```
DELIMITER //
CREATE PROCEDURE apply_dean_list(
    IN p_min_gpa DECIMAL(3,2),
    OUT p_count INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_count = -1;
    END;

    START TRANSACTION;

    UPDATE student SET dean_list = 1
    WHERE gpa >= p_min_gpa AND dean_list = 0;

    SET p_count = ROW_COUNT();  -- Number of rows updated

    INSERT INTO audit_log (action, details, action_date)
    VALUES ('DEAN_LIST',
            CONCAT(p_count, ' students added'),
            NOW());

    COMMIT;
END //
DELIMITER ;
```

Part 6: SQLite Equivalents

SQLite — No Stored Routines

SQLite does **not** support:

- ❌ CREATE FUNCTION
- ❌ CREATE PROCEDURE
- ❌ DELIMITER
- ❌ DECLARE / SET
- ❌ IN / OUT / INOUT parameters

SQLite — No Stored Routines

SQLite alternatives:

- ✅ Views for reusable queries
- ✅ CTEs for computed values
- ✅ CASE expressions for logic
- ✅ Application-level functions
- ✅ Triggers for automated actions

SQLite: View as "Reusable Function"

```sql
-- Instead of CREATE FUNCTION get_letter_grade()
-- Use a View or CASE directly:

CREATE VIEW student_grades AS
SELECT student_id, first_name, last_name, gpa,
    CASE
        WHEN gpa >= 3.7 THEN 'A'
        WHEN gpa >= 3.3 THEN 'B+'
        WHEN gpa >= 3.0 THEN 'B'
        WHEN gpa >= 2.7 THEN 'C+'
        WHEN gpa >= 2.0 THEN 'C'
        WHEN gpa >= 1.0 THEN 'D'
        ELSE 'F'
    END AS letter_grade
FROM student;

SELECT * FROM student_grades WHERE letter_grade = 'A';
```

SQLite: CTE as "Stored Procedure"

```sql
-- Instead of CALL get_dept_stats(1, @cnt, @avg, @max)
-- Use a CTE:

WITH dept_stats AS (
    SELECT dept_id,
           COUNT(*) AS student_count,
           ROUND(AVG(gpa), 2) AS avg_gpa,
           MAX(gpa) AS max_gpa
    FROM student
    WHERE dept_id = 1
    GROUP BY dept_id
)
SELECT * FROM dept_stats;
```

SQLite: Trigger as "Procedure"

```sql
-- Instead of a procedure for enrollment validation
CREATE TRIGGER validate_enrollment
    BEFORE INSERT ON enrollment
BEGIN
    SELECT CASE
        WHEN (SELECT COUNT(*) FROM enrollment
              WHERE student_id = NEW.student_id
                AND course_id = NEW.course_id) > 0
        THEN RAISE(ABORT, 'Student already enrolled')
    END;
END;
```

```sql
-- Automatic validation on insert
INSERT INTO enrollment (student_id, course_id)
VALUES (101, 5);  -- Error if already enrolled
```

SQLite: Application-Level Functions (Python)

```python
import sqlite3

def get_letter_grade(gpa):
    if gpa >= 3.7: return 'A'
    elif gpa >= 3.3: return 'B+'
    elif gpa >= 3.0: return 'B'
    elif gpa >= 2.0: return 'C'
    else: return 'F'

conn = sqlite3.connect('university.db')
conn.create_function('letter_grade', 1, get_letter_grade)

cursor = conn.execute('''
    SELECT first_name, gpa, letter_grade(gpa)
    FROM student ORDER BY gpa DESC
''')
for row in cursor:
    print(row)
```

Comparison Summary

MySQL	SQLite Equivalent
`CREATE FUNCTION`	View + CASE / Application function
`CREATE PROCEDURE`	CTE / Script
`IN` parameter	WHERE clause value
`OUT` parameter	SELECT result
`DELIMITER`	Not needed
`CALL proc()`	Run the CTE/Script
`DETERMINISTIC`	Not applicable

Part 7: Practice Exercises

Happy Coding :)

Exercise 1: Grade Calculator Function

MySQL (Theory):

Write a function `calculate_final_grade` that takes three parameters:

- `midterm DECIMAL(5,2)`
- `final_exam DECIMAL(5,2)`
- `project DECIMAL(5,2)`

Returns: Weighted average (midterm 30%, final 30%, project 40%)

SQLite (Practice):

Write a CTE-based query that calculates the same weighted grade.

Exercise 2: Enrollment Procedure

MySQL (Theory):

Write a procedure `batch_enroll` that:

- Takes IN `p_course_id`
- Enrolls all students from a specific department
- Uses OUT `p_enrolled_count`
- Uses error handling with HANDLER

SQLite (Practice):

Write an INSERT...SELECT that accomplishes the same task.

Exercise 3: Discount System

MySQL (Theory):

Write a function `calculate_total` that:

- Takes `p_order_id INT`
- Reads from `order_item` table (NOT DETERMINISTIC)
- Applies a 10% bulk discount if total > $100
- Returns the final total

SQLite (Practice):

Write a view that shows order totals with discount applied.

Exercise 4: Classification

Identify each as Function or Procedure, and DETERMINISTIC or NOT:

#	Description	F/P?	DET?
1	Converts Celsius to Fahrenheit	?	?
2	Inserts a new student record	?	?
3	Returns count of students in dept	?	?
4	Calculates area of a circle	?	?
5	Updates all GPA values by curve	?	?
6	Returns current semester name	?	?

Exercise 4: Answers

#	Description	F/P?	DET?
1	Celsius to Fahrenheit	**F**	**Yes**
2	Inserts a new student	**P**	N/A
3	Count students in dept	**F**	**No**
4	Area of circle	**F**	**Yes**
5	Updates all GPAs	**P**	N/A
6	Current semester name	**F**	**No**

📝 Homework

1. **MySQL Theory:**
   - Write a DETERMINISTIC function `bmi_calculator(weight_kg, height_m)`
   - Write a NOT DETERMINISTIC function `dept_rank(student_id)` that returns the student's rank within their department
   - Write a procedure `transfer_student(p_sid, p_new_dept, OUT p_msg)`
2. **SQLite Practice:**
   - Create a view `student_scholarship` showing students and their computed scholarship amounts
   - Write a CTE-based "department report" that shows dept stats, top student, and course count

🧠 Key Exam Concepts

Topic	What to Remember
Function vs Procedure	Function returns value, Procedure uses OUT
DELIMITER	Required for multi-statement routines
RETURNS vs RETURN	RETURNS = type, RETURN = value
IN / OUT / INOUT	Direction of data flow
DETERMINISTIC	Same input → same output (no tables, no NOW())
CALL	Used to invoke procedures, not functions
SELECT INTO	Assigns query result to a variable
ROW_COUNT()	Returns number of affected rows

Next Week Preview

Week 12: Cursors

- What are cursors and when to use them
- DECLARE / OPEN / FETCH / CLOSE lifecycle
- NOT FOUND handler pattern
- Row-by-row processing
- Cursors vs set-based operations
- SQLite alternatives for cursor patterns

Thank You!

Questions?

fkiran@yildiz.edu.tr

Required Reading: MySQL Function & Procedure Documentation